arXiv:2307.14936v1 [cs.CL] 27 Jul 2023

PANGU-CODER2: BOOSTING LARGE LANGUAGE MODELS FOR CODE WITH RANKING FEEDBACK

A PREPRINT

Bo Shen[*] **Jiaxin Zhang**[*] **Taihong Chen**[*] **Daoguang Zan**[§] **Bing Geng**[*] **An Fu**[*] **Muhan Zeng**[*]
Ailun Yu[†] **Jichuan Ji**[*] **Jingyang Zhao**[*] **Yuenan Guo**[*] **Qianxiang Wang**[*]
[*]Huawei Cloud Co., Ltd.
[§]Chinese Academy of Science
[†]Peking University

ABSTRACT

Large Language Models for Code (Code LLM) are flourishing. New and powerful models are released on a weekly basis, demonstrating remarkable performance on the code generation task. Various approaches have been proposed to boost the code generation performance of pre-trained Code LLMs, such as supervised fine-tuning, instruction tuning, reinforcement learning, etc. In this paper, we propose a novel **RRTF** (**R**ank **R**esponses to align **T**est&**T**eacher **F**eedback) framework, which can effectively and efficiently boost pre-trained large language models for code generation. Under this framework, we present PanGu-Coder2, which achieves 62.20% pass@1 on the OpenAI HumanEval benchmark. Furthermore, through an extensive evaluation on CoderEval and LeetCode benchmarks, we show that PanGu-Coder2 consistently outperforms all previous Code LLMs.

Keywords Large Language Model · Code Generation · Reinforcement Learning · Instruction Tuning

1 Introduction

As one of the most promising applications of large language model (LLM), code large language models have captivated considerable attention across academia and industry due to their remarkable capability in code-related tasks Zan et al. [2023]. Since OpenAI released Codex Chen et al. [2021], AlphaCode Li et al. [2022], PaLM-Coder Chowdhery et al. [2022], and PanGu-Coder Christopoulou et al. [2022] are subsequently published but in a closed-source way. Researchers open-source CodeParrot Huggingface [2021], PolyCoder Xu et al. [2022], PyCodeGPT Zan et al. [2022a], and SantaCoder Allal et al. [2023], but they fall far behind commercial models in terms of model size, capability, and performance. The situation is changed by Hugging Face[1], as the BigCode community releases StarCoder Li et al. [2023]: a 15B parameter model with 8K window size and FIM (Fill In the Middle, or infilling) capability. StarCoder outperforms many previous open-source large language models that support generating code from natural language descriptions, and even matches the OpenAI code-cushman-001 model on the HumanEval Chen et al. [2021] and MBPP benchmarks Austin et al. [2021].

However, most large language models for code still fall behind the latest commercial models like GPT-3.5 and GPT-4 from OpenAI OpenAI [2023], Bubeck et al. [2023]. We use Code LLM to denote the large language model majorly pre-trained on code corpus, like PanGu-Coder Christopoulou et al. [2022], Replit [2], and StarCoder Li et al. [2023]. Compared with open-source Code LLMs, the OpenAI GPT-family models are usually bigger in size and majorly pre-train on natural language corpus (with a small proposition of code-related data), which can contribute to their superior natural language comprehension and instruction following capabilities. Some efforts have been made to boost Code LLMs, like data engineering (phi-1 Gunasekar et al. [2023]), instruction tuning (WizardCoder Luo et al. [2023]), retrieval-augmented generation (ReAcc Lu et al. [2022], RepoCoder Zhang et al. [2023], etc.), and reinforcement learning (RLTF Liu et al. [2023], CodeRL Le et al. [2022], PPOCoder Shojaee et al. [2023], etc.).

[1]`https://huggingface.co`
[2]`https://github.com/replit/ReplitLM`

Although reinforcement learning (RL) seems to be a promising direction since programming is essentially a trial-and-error procedure, existing RL-based approaches face several major limitations. The motivation is intuitive and straightforward: as we expect the model to generate code according to human intent and requirements, reinforcement learning on Code LLMs can help the model enhance the ability to interpret and respond to code generation instructions, thus increasing the likelihood of generating a code to successfully solve a given problem. Typically, existing RL-based approaches design value/reward functions according to feedback signals from code processors, like compilers, debuggers, executors, and test cases. However, this leads to three limitations: First, regarding the test results as a reward directly provides limited improvements to the base model. Second, the adopted RL algorithm (like PPO) is complicated to implement and hard to train on large language models Liu et al. [2023]. Besides, running tests while training the model is time-consuming. As a result, previous works Le et al. [2022], Liu et al. [2023] only experiment on modestly-sized models, and the improvement is rather limited.

To address the problem of existing RL-based approaches and further exploit the potential of Code LLM, we propose the **RRTF** (**R**ank **R**esponses to align **T**est&**T**eacher **F**eedback) framework, which is a novel work to successfully apply natural language LLM alignment techniques on Code LLMs. Different from previous works like CodeRL Le et al. [2022] and RLTF Liu et al. [2023], we follow the idea of RLHF (Reinforcement Learning from Human Feedback) that empowers InstructGPT/ChatGPT Ouyang et al. [2022a], but implement a much simpler and efficient training approach using ranking responses as feedback instead of the absolute value of a reward model.

As a proof of concept, we apply RRTF on StarCoder 15B, and present a model that achieves the best performance among all published Code LLMs, namely the PanGu-Coder2. Through extensive evaluation on three benchmarks, including HumanEval, CoderEval, and LeetCode, we conjecture that Code LLMs do have the potential to surpass natural language models of the same or larger sizes on the code generation task. Furthermore, by analyzing the training process and manually inspecting the generation code samples, we highlight the importance of high-quality data in improving the models' instruction following and code writing capabilities.

In a nutshell, we make the following contributions:

- We introduce a new optimization paradigm named RRTF, which is a data-efficient, easy-to-implement, and model-agnostic framework to effectively boost the code generation performance of pre-trained Code LLMs.
- We present PanGu-Coder2, a model that improves nearly 30% over its base model and achieves new state-of-the-art performance on the HumanEval, CoderEval, and LeetCode benchmarks, surpassing all previously published Code LLMs.
- We share our experience and findings in constructing effective training data, training the model with RRTF, and optimizing such a model for fast inference.

2 Related Work

2.1 Large Language Model for Code (Code LLMs)

As a momentous milestone, Codex Chen et al. [2021] boasting a 12-billion-parameters model demonstrates the extraordinary capability to tackle up to 72% of Python programming problems. Subsequently, a new wave of code generation models, such as AlphaCode Li et al. [2022], PaLM-Coder Chowdhery et al. [2022], and PanGu-Coder Christopoulou et al. [2022], also were proposed. Despite the remarkable prowess exhibited by the aforementioned models, it is disheartening to note their unavailability as open-source projects. Therefore, several open-source code generation models, including CodeParrot Huggingface [2021], PolyCoder Xu et al. [2022], PyCodeGPT Zan et al. [2022a], SantaCoder Allal et al. [2023], and StarCoder Li et al. [2023], were released, injecting fresh vigor into the realm of code generation Chen et al. [2022]. Meanwhile, code generation models have also been applied to a broader range of practical coding scenarios. For example, CodeGeeX Zheng et al. [2023], BLOOM Scao et al. [2022] and ERNIE-Code Chai et al. [2022] have been proposed to facilitate multilingual modeling; JuPyT5 Chandel et al. [2022] is trained on a large corpus of Jupyter notebooks, aiming to elevate the experience of interactive programming; DocCoder Zhou et al. [2023a] and APICoder Zan et al. [2022b] have been proposed to empower language models with the ability to invoke APIs; Some models such as InCoder Fried et al. [2023], FIM Bavarian et al. [2022], MIM Nguyen et al. [2023], SantaCoder Allal et al. [2023], and StarCoder Li et al. [2023] support the code generation at arbitrary positions.

Of late, some efforts Zhou et al. [2023b], Peng et al. [2023] using the instruction tuning technique unlock the potential valuable knowledge stored within large language models, by fine-tuning on meticulously curated high-quality instruction datasets. In the field of code generation, WizardCoder 15B Luo et al. [2023] and phi-1 1.3B Gunasekar et al. [2023] achieve exceptional code generation performance by fine-tuning on the data generated by OpenAI's GPT-3.5 or GPT-4.

2.2 Reinforcement Learning on LLM

Reinforcement Learning from Human Feedback Large language models can generate untruthful, unexpected, and unhelpful outputs, which are not aligned with the intention of the end users. To align the behavior of large language models with human intentions, Ouyang et al. [2022b] proposed Reinforcement Learning from Human Feedback(RLHF) recently. The underlying idea is to leverage human preferences on given tasks to improve the behavior of a language model. A typical RLHF procedure consists of three steps, including supervised fine-tuning (SFT) which collects human demonstrations of desired model behavior and fine-tunes a language model, reward model (RM) training which employs humans to label the preferred output among various model outputs and trains a reward model based on the labeled data, and reinforcement learning via proximal policy optimization (PPO) which optimizes the language model against the reward model. OpenAI's GPT-3.5 and GPT-4 are trained with RLHF and their success demonstrates the effectiveness of RLHF to align the behavior of language models with human preferences. However, implementing RLHF requires heavy training resources and complex parameter tuning, which alleviates the technique from being easily applied in practice. In addition, the inefficiency and instability of RL algorithms can pose challenges to the alignment of language models. Given the limitations of heavy training resources and complex parameter tuning, Yuan et al. [2023] proposed the RRHF paradigm which leverages outputs with human preferences collected from various resources to train a model that aligns with human preferences. Its principle to align the model behavior to humans is to train a model to learn the outputs with better rewards based on human preferences among a set of outputs. Compared with RLHF, RRHF can be easily scaled to LLMs with larger sizes under a resource-constrained scenario. In view of the inefficiency and instability problem, Dong et al. [2023] proposed the reward-ranked fine-tuning (RAFT) technique for language models. Their underlying idea is to first select high-quality outputs of the model based on the output ranking estimated by a reward model and then leverage the selected outputs to train a model that aligns with human preferences. Compared with RLHF, the SFT-style RAFT typically converges faster than the PPO used in RLHF, while utilizing simpler parameter configuration and fewer computational resources.

Reinforcement Learning on Code The successful practice of RLHF has inspired researchers to improve the capability of Code LLMs with reinforcement learning. For example, CodeRL Le et al. [2022] integrates actor-critic RL framework with unit test signals to fine-tune models. Following CodeRL, PPOCoder Shojaee et al. [2023] uses the Proximal Policy Optimization (PPO) algorithm, but results in little improvements on the MBPP benchmark. Very recently, RLTF Liu et al. [2023] moves a step forward by adopting an online RL framework with multi-granularity unit test feedback, to overcome the limitation of offline RL adopted by CodeRL and PPOCoder.

2.3 Fine-tuning Code LLM

Fine-tuning on pre-trained language models is a mainstream modeling paradigm that maximizes the performance at downstream tasks. In the field of code, several works also adopt the paradigm to address code-related scenarios. For instance, CodeGen Nijkamp et al. [2022] and StarCoder Li et al. [2023] start by pre-training on a multilingual code corpus, followed by fine-tuning on monolingual data, thereby achieving superior performance on monolingual tasks. Codex-S Chen et al. [2021] and PanGu-Coder-FT Christopoulou et al. [2022] elevate their code generation capabilities by fine-tuning on competitive programming problems. Recently, instruction tuning Ouyang et al. [2022a], OpenAI [2023], as a form of supervised fine-tuning (SFT), is proposed to align the model with human behavior by learning abundant high-quality instruction corpus. In this regard, WizardCoder Luo et al. [2023] was fine-tuned on a series of instruction corpora derived from a teacher model, effectively maximizing its code knowledge with relatively limited parameters. In this technical report, PanGu-Coder2 employs ranking feedback strategy Yuan et al. [2023] during the fine-tuning process, and achieves surprising code generation performance.

3 Approach

3.1 Overview

In this technical report, we present a simpler but powerful framework **RRTF**, which seamlessly combines several cutting-edge techniques, including instruct tuning Peng et al. [2023], Evol-Instruct method Xu et al. [2023], Luo et al. [2023], and reinforcement learning Yuan et al. [2023]. The core idea of our approach is to guide a model towards producing higher-quality code, by utilizing the test signals and human preferences jointly as feedback to rank responses. Inspired by recent progress in reinforcement learning and instruction fine-tuning on top of large natural language models, especially the RLHF Ouyang et al. [2022a], RRHF Yuan et al. [2023], and RLTF Liu et al. [2023], we propose a new training paradigm, namely the RRTF framework. Figure 1 shows the overview of the RRTF framework, which consists of three steps: sampling, ranking, and training. In the sampling stage, responses are sampled with prompts generated via Evol-Instruct. In the ranking stage, responses from different sources are ranked according to unit tests

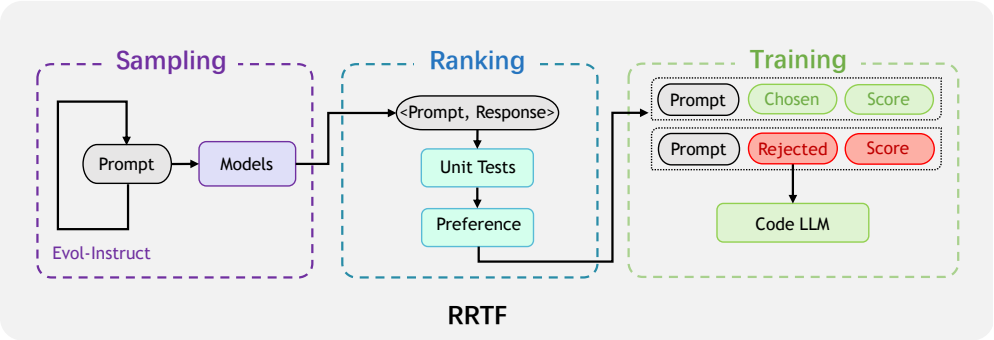

Figure 1: Overview of the proposed RRTF framework.

and heuristic preferences. In the training stage, triples of prompt and chosen/rejected responses with corresponding scores are used to train the Code LLM.

3.2 Model Architecture

In this work, we train a 15B parameter model based on the decoder-only Transformer with Multi-Query-AttentionShazeer [2019] and learned absolute positional embeddings. At the same time, FlashAttention is used to reduce the amount of calculation and memory usage. Hence, the max length of the model can be scaled to 8192. Tabel 1 shows the detailed hyper-parameters of our model.

Table 1: The hyper-parameters of our model

Hyper-Parameters	Value
Hidden size	6144
Max Length	8192
Num of attention heads	48
Num of transformer hidden layers	40

3.3 Training Corpus

We follow the Evol-Instruct technique Xu et al. [2023], Luo et al. [2023] to construct our training corpus, since manually collecting a high-quality corpus is labor-intensive and time-consuming. Specifically, we started from Alpaca 20K dataset[3] and iteratively evolve the programming problems in this dataset via in-depth evolving to obtain new programming problems (the prompt is shown in Figure 2). With these problems, we sampled answers from different models. In total, we collected an initial corpus containing 100K programming problems with answers, which we refer to as instruction and solution pairs. In addition, we conducted data preprocessing on our initial corpus using several manually-defined rules and reduced the size of the corpus to 68K. More importantly, to prevent data leakage, we devoted considerable efforts to surveying the potential overlap between the collected 68K dataset and the HumanEval benchmark. After conducting a meticulous survey, we confirm that there is no data leakage in our experiments, further validating the effectiveness of PanGu-Coder2.

3.4 RRTF framework

Inspired by RRHF Yuan et al. [2023], we propose the **RRTF** (**R**ank **R**esponses to align **T**est&**T**eacher **F**eedback) framework for Code LLMs. RRHF [4] is proposed as a simplified training paradigm for RLHF, which ranks responses from different sources according to human preferences, and aligns the model through a ranking loss function. Compared with RLHF, RRHF can efficiently align the output probabilities of a language model with human preferences, with only 1-2 models required during the tuning period, and it is simpler than PPO in terms of implementation, hyperparameter tuning, and training.

[3] `https://huggingface.co/datasets/sahil2801/CodeAlpaca-20k`
[4] `https://github.com/GanjinZero/RRHF`

I want you to act as a Programming Contest Designer. Your objective is to rewrite a programming task based on the given task by increasing the difficulty a bit.
You can increase the difficulty using, but not limited to, the following methods:
{methods}

Your response is the rewritten programming task (#Rewritten Task#).
The #Rewritten Task# must be reasonable and must be understood and responded by humans, and also solvable with code. It should not be dependent on the #Given Task#. Your rewriting cannot omit the non-text parts such as the table and code in #Given Task#:. Also, please do not omit the input in #Given Task#.
*The rewritten task and the given task should have the similar length. **
*The rewritten task should ask for a function-level code solution.**
"#Given Task#", "#Rewritten Task#", "given task", and "rewritten task" are NOT allowed to appear in #Rewritten Task#.
#Given Task#
{instruction}
#Rewritten Task#

Figure 2: Prompt to evolve over the CodeAlpaca dataset.

Instead of aligning the model with human intents, the purpose of code generation is to improve generating correctness, so we replace the **H** (human) with **T**, which can be a combination of tests and teachers (more powerful models or human experts), they can jointly form a feedback signal to guide the generation of Code LLM and most of the feedback can be fully- or semi-automatically obtained in the faster way. The training procedures of RRTF can be divided into 3 steps:

1. **Step1: Sampling** In the sampling stage, responses are sampled with prompts. Based on the prompts generated by the Evol-Instruct (see Section 3.3), we sample the responses both from the student model (model to train) and teacher models by various temperatures. The process is offline and in parallel, so we can efficiently get enough samples for training.
2. **Step2: Ranking** In the ranking stage, responses from different sources are ranked according to unit tests and heuristic preferences. After obtaining all responses, we extract the programs from the responses and execute them in a running environment that supports large-scale parallel execution. According to the test results, there are 4 situations, which are *compiled error*, *runtime error*, *pass partial tests*, *all pass*. For each data, we assign different scores from low to high based on the above situations. Meanwhile, we filter out data whose teachers' score is lower than the student model. For two samples that fall into the same situation, we always assign a higher rank to the sample from the teachers, since we *prefer* the student to learn from the teacher.
3. **Step3: Training** In the training stage, triples of prompt and chosen/rejected responses with corresponding scores are used to train the Code LLM. During training, for each prompt x, we have a pair of response $\{y_{tea}, y_{stu}\}$, where y_{tea} is the response generated by the teachers, and y_{stu} is the response generated by the student model. So we can indicate the conditional log probability(length-normalized) p_i by:

$$p_i = \frac{\sum_t \log P_\pi \left(y_{i,t} \mid x, y_{i,<t}\right)}{\|y_i\|}$$

where π is the model, $i \in \{tea, stu\}$, t is the time step. And the rank loss can be expressed as:

$$L_{rank} = -\sum_{r_{tea} > r_{stu}} \left(r_{tea} - r_{stu}\right) \min\left(0, p_{tea} - p_{stu}\right)$$

where r_{tea} and r_{stu} are the scores given in ranking stage. There is also a cross-entropy loss similar to supervised fine-tuning, which lets the model learn the response generated by the teacher:

$$L_{ft} = -\sum_t \log P_\pi \left(y_{tea,t} \mid x, y_{tea,<t}\right)$$

Finally, the total loss is the sum of the above two losses:

$$L = L_{rank} + L_{ft}$$

3.5 Implementation Details

We choose the StarCoder 15B Li et al. [2023] as the base model, and train it with a global batch size of 512 for 6 epochs.

Figure 3 shows the format of one single training sample. In addition to adding a pair of triple quotation marks on the prompt, we only use the code snippets extracted from responses for training.

```
"""
Write a Python script to create a webserver that can respond to "GET" requests from a web browser.
"""
import http.server
import socketserver

PORT = 8080
Handler = http.server.SimpleHTTPRequestHandler

with socketserver.TCPServer(("", PORT), Handler) as httpd:
    print("Serving at port", PORT)
    httpd.serve_forever()
```

Figure 3: Example data format of the training sample.

4 Evaluation

We have conducted an extensive evaluation to study the performance of our approach. This section describes the settings of our evaluation and reports the experimental results as well as our findings.

4.1 Evaluation Setup

4.1.1 Main Evaluated Models

- **CodeGen-mono 16B** Nijkamp et al. [2022] is a variant of CodeGen-Multi 16B, specifically fine-tuned using additional Python code from GitHub.
- **CodeGeeX 13B** Zheng et al. [2023] is a multilingual language model for code with a parameter count of 13B, which is trained on approximately 850B tokens from 23 programming languages.
- **StarCoder 15B** Li et al. [2023] is a Code LLM with 15B parameters and a context size of 8K, which supports infilling capabilities and fast inference.
- **CodeT5+ 16B** Wang et al. [2023], an encoder-decoder Code LLM, boasts modular flexibility, accommodating diverse code-related downstream tasks.
- **WizardCoder 15B** Luo et al. [2023] is the state-of-the-art Code LLM prior to PanGu-Coder2, and is trained using the Evol-Instruct technique.

4.1.2 Benchmarks

- HumanEval:[5] Released alongside Codex by OpenAI Chen et al. [2021], the most widely-adopted benchmark for comparing LLMs' performance on code generation. HumanEval consists of 164 manually-written programming tasks.
- CoderEval Yu et al. [2023]: A pragmatic code generation benchmark to evaluate models under realistic software development scenarios, including 230 functions with tests from 43 open-source Python projects.
- LeetCode (after 2022.7): We collected problems from leetcode that meet the following criteria:
  - Problems that are publicly available and can be accessed for free.
  - Problems that were created after July 1st, 2022, which ensures that any data in this benchmark does not overlap with the training data of StarCoder, which only consists of code before June 2022.

  Besides the problem description, we also collected Python editor hints including method name and signature. We took editor hints as prompt input and tested models' output using public tests. As a result, this benchmark

[5] https://github.com/openai/human-eval

includes a total of 300 problems(with problem id ≥ 2325), including 79 easy problems, 150 medium problems, and 71 hard problems.

4.1.3 Metric

Pass@k Same as related works, we also adopt the *pass@k* metric implemented by OpenAI Chen et al. [2021] to assess the functional correctness of generated code, where $n(n \geq k)$ code samples are generated for each problem, and the number of correct samples c is counted. The functional correctness of a code sample is determined by executing the corresponding unit tests and checking if it passes all test cases. Given the total number of generation n, the number of correct samples c, and the sampling budget k, *pass@k* is calculated via the unbiased estimator:

$$pass@k := \mathbb{E}[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}], n = 200, k \in \{1, 10, 100\}$$

4.1.4 Decoding Strategy

For experiments that evaluate the performance of models on code generation by estimating pass@k, we used a temperature of 0.2 to generate responses for pass@1, and a temperature of 1.2 for pass@10 and pass@100 for more diversity. For closed-source models, we retrieved the data from previous papers. For available models, we generated 200 samples to guarantee a statistically reliable result as much as possible. Additionally, we used a top_p of 0.95 for nucleus sampling. For comparison of PanGu-Coder2 with other latest open-source models on three benchmarks, we used the greedy decoding strategy.

4.1.5 Prompts

We noticed that the performance of a Code LLM could be largely affected by the prompt used for generating solutions to a programming problem. To maintain consistency with existing studies, for a given Code LLM, we leveraged the prompt reported in its corresponding paper to conduct our evaluation. The detailed code generation prompt for PanGu-Coder2 and other models are as follows:

Prompt for PanGu-Coder2
"""
{docstring}
"""
{function signature}

Prompt for StarCoder
{function signature}
    """
    {docstring}
    """

Prompt for WizardCoder
Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.
Instruction:
Create a Python Script for this problem:
{function signature}
    """
    {docstring}
    """

Response:

Table 2: Results of pass@1/10/100 of well-known models on HumanEval. Most scores are retrieved from previous papers as they are reported. For PanGu-Coder2, we follow the Codex Chen et al. [2021] and AlphaCode Li et al. [2022] paper to generate n=200 samples and report the optimal pass@1/10/100 when temperature=0.2/1.2/1.2 and top_p=0.95. The same settings are used for StarCoder and WizardCoder (marked with *).

Model	Params	Pass@k (%)		
		k=1	k=10	k=100
Closed-source Models				
AlphaCode Li et al. [2022]	1.1B	17.1	28.2	45.3
Phi-1 Gunasekar et al. [2023]	1.3B	50.6	-	-
Codex Chen et al. [2021]	12B	28.81	46.81	72.31
LaMDA Thoppilan et al. [2022]	137B	14.0	-	47.3
PaLM-Coder Chowdhery et al. [2022]	540B	36.0	-	88.4
GPT-3.5 OpenAI [2023]	-	48.1	-	-
GPT-3.5 Luo et al. [2023]	-	68.9	-	-
GPT-4 OpenAI [2023]	-	67.0	-	-
GPT-4 Bubeck et al. [2023]	-	82.0	-	-
Open-source Models				
CodeGen-mono Nijkamp et al. [2022]	16B	29.28	49.86	75.00
CodeGeeX Zheng et al. [2023]	13B	22.89	39.57	60.92
StarCoder Li et al. [2023]*	15B	33.60	45.78	79.82
CodeT5+ Wang et al. [2023]	16B	30.9	51.6	76.7
WizardCoder Luo et al. [2023]*	15B	57.30	73.32	90.46
PanGu-Coder2*	15B	**61.64**	**79.55**	**91.76**

4.2 Evaluation Results

4.2.1 Performance

We compared PanGu-Coder2 with existing Code LLMs in terms of Python code generation performance. Table 2 shows the comparison result of pass@k on the HumanEval benchmark. Across all open-source models, PanGu-Coder2 achieves the best results for all k values (pass@1=61.64, pass@10=79.55, pass@100=91.76). Compared with WizardCoder which was the state-of-the-art Code LLM on the HumanEval benchmark, we can observe that PanGu-Coder2 outperforms WizardCoder by a percentage of 4.34%. With regard to StarCoder, we can observe 28% absolute improvement in terms of pass@1 score (from 33.6% to 61.6%). In addition, for pass@10 and pass@100, the performance of PanGu-Coder2 is consistently better than that of StarCoder.

Across all closed-source models, PanGu-Coder2 attains the second position. Compared with larger models including PaLM-Coder and LaMDA, PanGu-Coder2 performs better despite being smaller in scale. Another promising observation is that PanGu-Coder2 outperforms OpenAI's GPT-3.5. However, there is still a gap between our model and OpenAI's GPT-4 (the version reported in OpenAI's report OpenAI [2023]).

Table 3 shows the comparison result of greedy decoding pass@1. Across all benchmarks, we can observe that PanGu-Coder2 achieves the best results among all models, with a pass@1 value of 62.20% on HumanEval, 38.26% on CoderEval, and 32/30/10 on LeetCode. A promising observation is that PanGu-Coder2 not only surpasses WizardCoder and StarCoder on HumanEval, but also outperforms these two models on CoderEval and LeetCode. This indicates that PanGu-Coder2 not only excels at simple programming tasks, but also performs outstandingly well on context-aware development tasks and programming contest problems.

From the experimental results shown in Tables 2 and 3, we can conclude that:

- PanGu-Coder2 achieves a state-of-the-art 61.64% pass@1 on HumanEval among open-source models.
- PanGu-Coder2 outperforms models of larger scale including PaLM-Coder and LaMDA despite being smaller in scale.
- PanGu-Coder2 is the only model we tested that achieves the best performance on HumanEval, CoderEval, and LeetCode at the same time.

Table 3: Performance comparison of *PanGu-Coder2* with previous models on three benchmarks by greedy decoding.

Model	Params	HumanEval (text2code)	CoderEval (context2code)	LeetCode (easy/medium/hard)
PanGu-Coder	2.6B	23.78	15.21	6/3/0
Replit-code-instruct-glaive[6]	2.7B	56.10	27.39	3/5/2
StarCoder	15B	32.93	37.82	18/13/2
WizardCoder	15B	59.80	33.48	29/22/7
PanGu-Coder2	15B	**62.20**	**38.26**	**32/30/10**

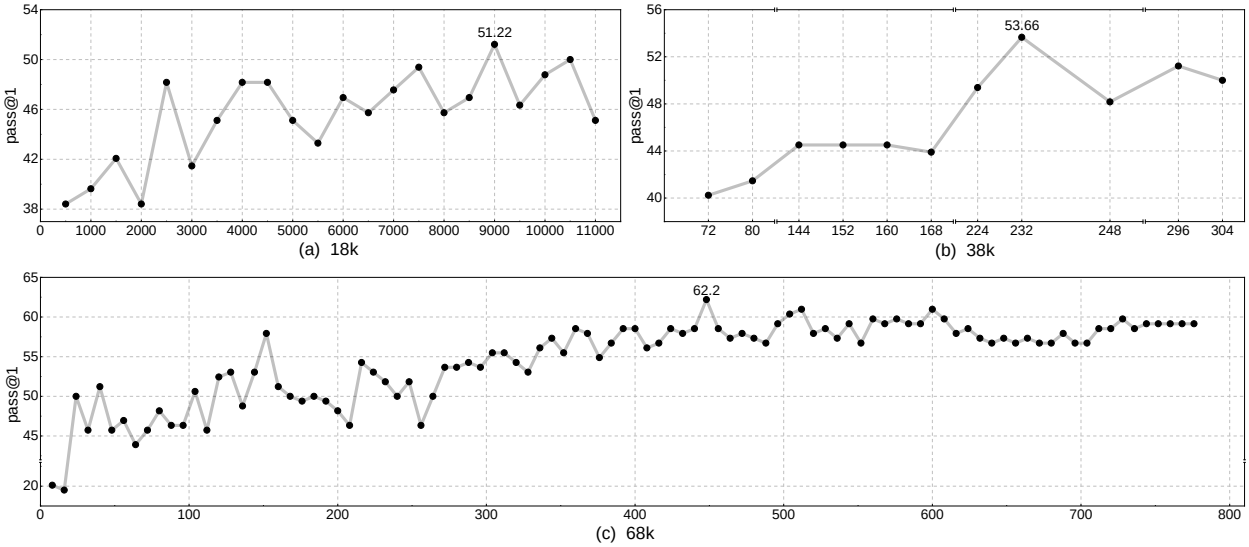

Figure 4: Performance change when in the training process (pass@1 on HumanEval with greedy decoding). The number of steps in an epoch for (a),(b), and (c) is roughly 2250, 74, and 132 respectively.

4.2.2 Findings

To analyze the training process of PanGu-Coder2, we focus on two of the key factors that affect the performance of large language models: the dataset size and the training compute.

Dataset size The overall accuracy (estimated via greedy decoding pass@1) increases along with the growth of dataset size, as shown in Figure 4. Also, as the size of the dataset grows, the training curve becomes more stable, at roughly 2-3 epochs on 38k/68k dataset. As for the 18k dataset, performance still oscillates drastically after 3 epochs. This suggests that more and variant corpus can result in better performance, while the training cost is still acceptable as epochs needed for reaching the best performance do not increase along with the scale of the corpus.

Training compute Regardless of dataset size, the accuracy may drop drastically or stay flat at the start of the training. After roughly 2 epochs, the training curve becomes more stable and the accuracy consistently increases as the loss decreases. The best performances are reached after 3 epochs while the accuracy becomes even more stable after 4 epochs, showing a sign of convergence. This suggests that the model needs roughly 3-4 epochs to fully capture the knowledge in the dataset, and training steps after that may have very little help towards increasing the model's capability.

4.2.3 Case Study

To empirically study the model and shed light on future work, we compare and analyze the successful and failed cases of three models: the base model StarCoder, the instruction-tuned model WizardCoder, and the PanGu-Coder2 model.

Figure 5 shows the difference and intersection of solved problems by three models, in terms of greedy decoding and nucleus sampling. From the figure, we find that PanGu-Coder2 and WizardCoder can be complementary: though PanGu-Coder2 solves the most problems and some of them cannot be solved by WizardCoder, there are problems that

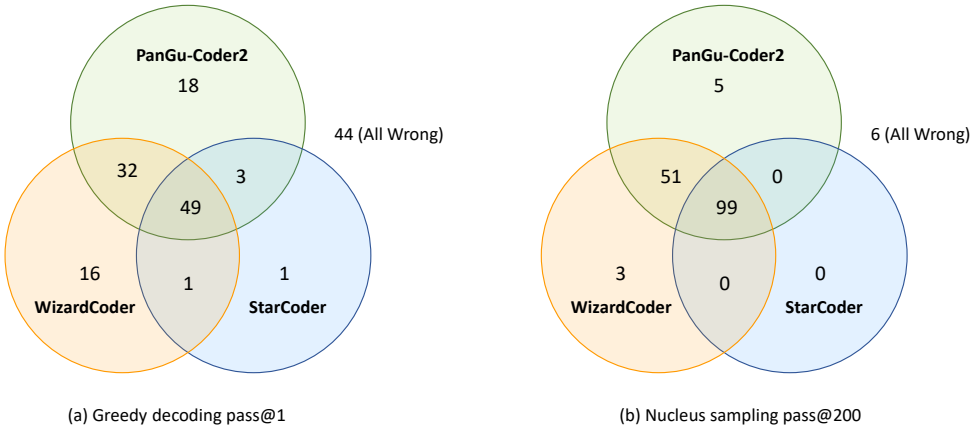

Figure 5: Numbers of correctly-solved problems by three models on HumanEval.

were only solved by WizardCoder, which boosts the StarCoder performance in the instruct tuning way. Besides, there are some problems that cannot be solved by any of these models, even sampling for 200 times.

We choose several representative example codes generated by StarCoder, WizardCoder, and PanGu-Coder2 as the case study, to conduct a critical evaluation of the strengths and weaknesses of PanGu-Coder2. As depicted in Figure 6, PanGu-Coder2 adeptly comprehends the logical aspects of programming problems, while WizardCoder and StarCoder fall short in this regard. This observation signifies that PanGu-Coder2 has effectively established an meticulous mapping between programming problem statements and code solutions via our proposed ranking feedback. According to the depiction in Figure 7, in certain instances, PanGu-Coder2 and StarCoder are outperformed by WizardCoder, which may benefit from training with extensive rich-comment code instructions. To a certain extent, this observation implies that incorporating step-by-step comment data may yield positive effects during the training process. In addition, Figure 8 shows a case where StarCoder, WizardCoder, and PanGu-Coder2 all give incorrect code solutions due to the intricacies and challenges posed by the programming problem. This observation indicates that currently code LLMs still fall short of human-like proficiency in tackling complex programming requirements, leaving some room for improvement.

Prompt

```
"""
Given a positive integer n, you have to make a pile of n levels of stones.
The first level has n stones.
The number of stones in the next level is:
    - the next odd number if n is odd.
    - the next even number if n is even.
Return the number of stones in each level in a list, where element at index i represents the number of stones in the level (i+1).

Examples:
>>> make_a_pile(3)
[3, 5, 7]
"""
```

PanGu-Coder2 ✓

```
def make_a_pile(n):
    pile = []
    next_num = n
    for i in range(n):
        pile.append(next_num)
        next_num += 2
    return pile
```

WizardCoder ✗

```
def make_a_pile(n):
    pile = [n]

    for i in range(1, n):
        if pile[-1] % 2 == 1:
            pile.append(pile[-1]+2)
        else:
            pile.append(pile[-1]+1)

    return pile
```

StarCoder ✗

```
def make_a_pile(n):
    pile = [n]
    for i in range(n-1
        if pile[i] % 2 != 0:
            pile.append(pile[i] + 1)
        else:
            pile.append(pile[i] + 2)
    return pile
```

Figure 6: A HumanEval case of PanGu-Coder2 surpassing WizardCoder and StarCoder.

Prompt

```
"""
Given an array arr of integers and a positive integer k, return a sorted list of length k with the maximum k numbers in arr.
Example 1:
    Input: arr = [-3, -4, 5], k = 3
    Output: [-4, -3, 5]
Example 2:
    Input: arr = [4, -4, 4], k = 2
    Output: [4, 4]
Example 3:
    Input: arr = [-3, 2, 1, 2, -1, -2, 1], k = 1
    Output: [2]
Note:
    1. The length of the array will be in the range of [1, 1000].
    2. The elements in the array will be in the range of [-1000, 1000].
    3. 0 <= k <= len(arr)
"""
```

PanGu-Coder2 ✗

```
def maximum(arr, k):
    arr.sort()
    return arr[-k:]
```

WizardCoder ✓

```
def maximum(arr, k):
    # Sort the array in descending order
to find the largest k numbers
    arr = sorted(arr, reverse=True)

    # Return a sorted list with the
first k elements in arr
    return sorted(arr[:k])
```

StarCoder ✗

```
def maximum(arr, k):
    # Time Complexity: O(nlogn)
    # Space Complexity: O(n)
    arr.sort()
    return arr[-k:]
```

Figure 7: A HumanEval case of WizardCoder surpassing PanGu-Coder2 and StarCoder.

Prompt

```
"""
You are given a list of two strings, both strings consist of open parentheses '(' or close parentheses ')' only.
Your job is to check if it is possible to concatenate the two strings in some order, that the resulting string will be good.
A string S is considered to be good if and only if all parentheses in S are balanced. For example: the string '(())()' is good,
while the string '())' is not.
Return 'Yes' if there's a way to make a good string, and return 'No' otherwise.

Examples:
match_parens(['()(', ')']) == 'Yes'
match_parens([')', ')']) == 'No'
"""
```

PanGu-Coder2 ✗

```
def match_parens(lst):
    stack = []
    for char in ''.join(lst):
        if char == '(':
            stack.append(char)
        elif char == ')':
            if not stack or stack.pop() != '(':
                return 'No'
    return 'Yes' if not stack else 'No'
```

WizardCoder ✗

```
def match_parens(lst):
    str1, str2 = lst
    stack = []
    for c in str1 + str2:
        if c == '(':
            stack.append(c)
        elif c == ')':
            if not stack:
                return "No"
            stack.pop()
    return "Yes" if not stack else "No"
```

StarCoder ✗

```
def match_parens(lst):
    stack = []
    for i in lst:
        if i == '(':
            stack.append(i)
        elif i == ')':
            if len(stack) == 0:
                return 'No'
            else:
                stack.pop()
    if len(stack) == 0:
        return 'Yes'
    else:
        return 'No'
```

Figure 8: A HumanEval case where PanGu-Coder2, WizardCoder, and StarCoder all generate incorrect outputs.

4.3 Inference Optimization

Since GPU memory consumption and inference speed are crucial factors for the deployment and use of model in practice, we conducted experiments involving the following quantization techniques to study the optimization strategies of model inference:

- CTranslate2:[7] CTranslate2 is a library for accelerating Transformer models when inference, developed by OpenNMT.
- GPTQ:[8] A LLMs quantization package based on GPTQ algorithm.

Table 4 shows the GPU memory consumption, inference speed, and HumanEval performance of models optimized using different quantization techniques. We used 8-bit (4-bit) quantization and the following decoding parameters in the inference stage: top_p=0.95, tempreture=0.2, max_new_tokens=512. Across all quantization techniques, we can observe a significant decrease in terms of memory usage and a significant increase in terms of inference speed. It is incredible that after being quantized with CTranslate2, the performance of our model on HumanEval has a slight improvement. A plausible reason for this phenomenon is the robustness of PanGu-Coder2 itself. We plan to conduct an in-depth study on this interesting result in our further work.

Table 4: A comparison of different quantization techniques (on the same device)

Model	Precision	GPU Memory Consumption (GB)	Inference Speed (ms/token)	HumanEval (greedy decoding)
PanGu-Coder2	float16	32.36	75	62.20
PanGu-Coder2-CTranslate2	int8	16.29	33	64.63
PanGu-Coder2-GPTQ	int8	16.92	51	51.22
PanGu-Coder2-GPTQ	int4	9.82	42	51.83

5 Conclusion

In this paper, we introduce a novel frameork, namely RRTF, and present a new Code LLM, namely PanGu-Coder2. Firstly, we adopt the Evol-Instruct technique to obtain a substantial amount of high-quality natural language instruction and code solution data pairs. Then, we train the base model by ranking candidate code solutions using feedback from test cases and heurstic preferences. Through comprehensive evaluations on HumanEval, CodeEval, and LeetCode benchmarks, PanGu-Coder2 achieves new state-of-the-art performance among billion-parameter-level Code LLMs, surpassing all of the existing ones by a large margin. In our future work, we will delve into the combination of RRTF and instruct tuning to boost the performance of Code LLMs.

References

Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Wang Yongji, and Jian-Guang Lou. Large language models meet NL2Code: A survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7443–7464, Toronto, Canada, July 2023. Association for Computational Linguistics. URL `https://aclanthology.org/2023.acl-long.411`.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom, Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de, Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey, Cherepanov, James Molloy, Daniel Jaymin Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de, Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378:1092 – 1097, 2022.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua

[7] `https://github.com/OpenNMT/CTranslate2`

[8] `https://github.com/PanQiWei/AutoGPTQ`

Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam M. Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Benton C. Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier García, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Díaz, Orhan Firat, Michele Catasta, Jason Wei, Kathleen S. Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling language modeling with pathways. *ArXiv*, abs/2204.02311, 2022.

Fenia Christopoulou, Gerasimos Lampouras, Milan Gritta, Guchun Zhang, Yinpeng Guo, Zhong-Yi Li, Qi Zhang, Meng Xiao, Bo Shen, Lin Li, Hao Yu, Li yu Yan, Pingyi Zhou, Xin Wang, Yu Ma, Ignacio Iacobacci, Yasheng Wang, Guangtai Liang, Jia Wei, Xin Jiang, Qianxiang Wang, and Qun Liu. PanGu-Coder: Program synthesis with function-level language modeling. *ArXiv*, abs/2207.11280, 2022.

Huggingface. Training CodeParrot from Scratch, 2021. `https://huggingface.co/blog/codeparrot`.

Frank F. Xu, Uri Alon, Graham Neubig, and Vincent J. Hellendoorn. A systematic evaluation of large language models of code. *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 2022.

Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. CERT: Continual pre-training on sketches for library-oriented code generation. In *International Joint Conference on Artificial Intelligence*, 2022a.

Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Muñoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alexander Gu, Manan Dey, Logesh Kumar Umapathi, Carolyn Jane Anderson, Yangtian Zi, J. Poirier, Hailey Schoelkopf, Sergey Mikhailovich Troshin, Dmitry Abulkhanov, Manuel Romero, Michael Franz Lappert, Francesco De Toni, Bernardo Garc'ia del R'io, Qian Liu, Shamik Bose, Urvashi Bhattacharyya, Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco Zocca, Sourab Mangrulkar, David Lansky, Huu Nguyen, Danish Contractor, Luisa Villa, Jia Li, Dzmitry Bahdanau, Yacine Jernite, Sean Christopher Hughes, Daniel Fried, Arjun Guha, Harm de Vries, and Leandro von Werra. SantaCoder: don't reach for the stars! *ArXiv*, abs/2301.03988, 2023.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023. doi:10.48550/arXiv.2303.08774. URL `https://doi.org/10.48550/arXiv.2303.08774`.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. WizardCoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023.

Shuai Lu, Nan Duan, Hojae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. Reacc: A retrieval-augmented code completion framework. *arXiv preprint arXiv:2203.07722*, 2022.

Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*, 2023.

Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Wei Yang, and Deheng Ye. Rltf: Reinforcement learning from unit test feedback, 2023.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven CH Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. *arXiv preprint arXiv:2207.01780*, abs/2207.01780, 2022.

Parshin Shojaee, Aneesh Jain, Sindhu Tipirneni, and Chandan K Reddy. Execution-based code generation using deep reinforcement learning. *arXiv preprint arXiv:2301.13816*, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022a.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.

Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shanshan Wang, Yufei Xue, Zi-Yuan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. CodeGeeX: A pre-trained model for code generation with multilingual evaluations on humaneval-x. *ArXiv*, abs/2303.17568, 2023.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. BLOOM: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

Yekun Chai, Shuohuan Wang, Chao Pang, Yu Sun, Hao Tian, and Hua Wu. ERNIE-Code: Beyond english-centric cross-lingual pretraining for programming languages. *arXiv preprint arXiv:2212.06742*, 2022.

Shubham Chandel, Colin B. Clement, Guillermo Serrato, and Neel Sundaresan. Training and evaluating a jupyter notebook data science assistant. *ArXiv*, abs/2201.12901, 2022.

Shuyan Zhou, Uri Alon, Frank F Xu, Zhengbao JIang, and Graham Neubig. DocCoder: Generating code by retrieving and reading docs. In *The Eleventh International Conference on Learning Representations*, 2023a.

Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Yongji Wang, and Jian-Guang Lou. When language model meets private library. In *Conference on Empirical Methods in Natural Language Processing*, 2022b.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *The Eleventh International Conference on Learning Representations*, 2023.

Mohammad Bavarian, Heewoo Jun, Nikolas A. Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *ArXiv*, abs/2207.14255, 2022.

Anh Nguyen, Nikos Karampatziakis, and Weizhu Chen. Meet in the Middle: A new pre-training paradigm. *arXiv preprint arXiv:2303.07295*, 2023.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. Lima: Less is more for alignment. *arXiv preprint arXiv:2305.11206*, 2023b.

Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022b. URL `http://papers.nips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html`.

Zheng Yuan, Hongyi Yuan, Chuanqi Tan, Wei Wang, Songfang Huang, and Fei Huang. Rrhf: Rank responses to align language models with human feedback without tears. *arXiv preprint arXiv:2304.05302*, 2023.

Hanze Dong, Wei Xiong, Deepanshu Goyal, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward ranked finetuning for generative foundation model alignment. *arXiv preprint arXiv:2304.06767*, 2023.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. CodeGen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023.

Noam Shazeer. Fast transformer decoding: One write-head is all you need. *CoRR*, abs/1911.02150, 2019. URL `http://arxiv.org/abs/1911.02150`.

Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*, 2023.

Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Tao Xie, and Qianxiang Wang. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. *arXiv preprint arXiv:2302.00288*, 2023.

Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.